The Automaton Auditor

Interim Report Submission

By:

Abdurezak Arage

Executive Summary

This report documents the architectural decisions, implementation progress, and remaining gaps in the development of the Automaton Auditor a hierarchical, LangGraph-based Digital Courtroom designed to autonomously evaluate AI-generated repositories. The system is structured around a Detective Layer for forensic evidence collection, a Judicial Layer for dialectical evaluation through adversarial personas, and a Chief Justice synthesis engine for deterministic conflict resolution.

The Detective Layer has been successfully implemented with typed state management, AST-based repository analysis, git forensic tooling, and sandboxed execution. The Judicial Layer and deterministic synthesis engine are partially scaffolded and will be completed before final submission.

1. Architecture Decisions

1.1 State Management & Typing Strategy

The Automaton Auditor uses Pydantic BaseModel classes and TypedDict-based graph state with Annotated reducers. This design ensures strict type safety, structured outputs, and safe parallel execution across multiple agents.

Key Decisions:

- Use Pydantic models for Evidence, JudicialOpinion, CriterionResult, and AuditReport.
- Use TypedDict for AgentState.
- Use operator.ior and operator.add reducers to prevent state overwriting.

This approach prevents the common failure mode of parallel agents overwriting shared state, which is explicitly penalized under the rubric's 'State Management Rigor' dimension.

1.2 Detective Layer Design (Forensic Sub-Agents)

The Detective Layer is responsible for collecting objective, structured Evidence objects. These agents do not provide opinions only verifiable facts.

RepoInvestigator:

- Uses sandboxed git clone via tempfile.
- Extracts commit history using git log --oneline --reverse.
- Uses Python AST parsing to analyze builder.add_edge() calls.

DocAnalyst:

- Implements RAG-lite PDF chunking.
- Queries for key theoretical concepts (Dialectical Synthesis, Fan-In/Fan-Out, Metacognition).
- Cross-references report claims with repository evidence.

1.3 Sandboxing & Security Strategy

All repository cloning is executed inside a temporary directory using tempfile.mkdtemp(). No raw os.system calls are used. Instead, subprocess.run is employed with proper error handling to prevent shell injection vulnerabilities.

This directly satisfies the 'Safe Tool Engineering' requirements of the rubric.

2. Current Implementation Status

2.1 Completed Components

- Parallel Detective Graph (Fan-Out / Fan-In implemented).
- Typed AgentState with reducers.
- AST-based graph structure analysis.
- Git forensic timeline extraction.
- PDF ingestion and keyword-based retrieval.

2.2 Partially Implemented / Pending

- Judge Nodes (Prosecutor, Defense, Tech Lead) persona prompts drafted but LLM structured enforcement pending.
- ChiefJustice deterministic synthesis rules scaffold exists, logic incomplete.
- VisionInspector planned but not yet integrated.

- Markdown serialization of AuditReport to be finalized.

3. Planned Judicial Layer & Dialectical Synthesis

3.1 Three-Persona Dialectical Model

The Judicial Layer implements a structured dialectical process. For every rubric criterion, three judges analyze identical evidence in parallel.

Prosecutor (Critical Lens):

- Assumes 'vibe coding'. Searches for orchestration fraud and hallucination liability.

Defense (Optimistic Lens):

- Rewards effort, architectural intent, and engineering iteration.

Tech Lead (Pragmatic Lens):

- Evaluates maintainability, architectural soundness, and practical viability.

All three judges must return structured JSON using .with_structured_output(JudicialOpinion). Freeform text responses will trigger retry logic.

3.2 Chief Justice Synthesis Engine

The Chief Justice applies deterministic Python logic to resolve score conflicts.

- Security Override Rule confirmed vulnerabilities cap score at 3.
- Fact Supremacy Rule Detective evidence overrides judge hallucinations.
- Functionality Weight Rule Tech Lead has highest weight for architecture criteria.
- Dissent Requirement variance > 2 requires explicit dissent summary.

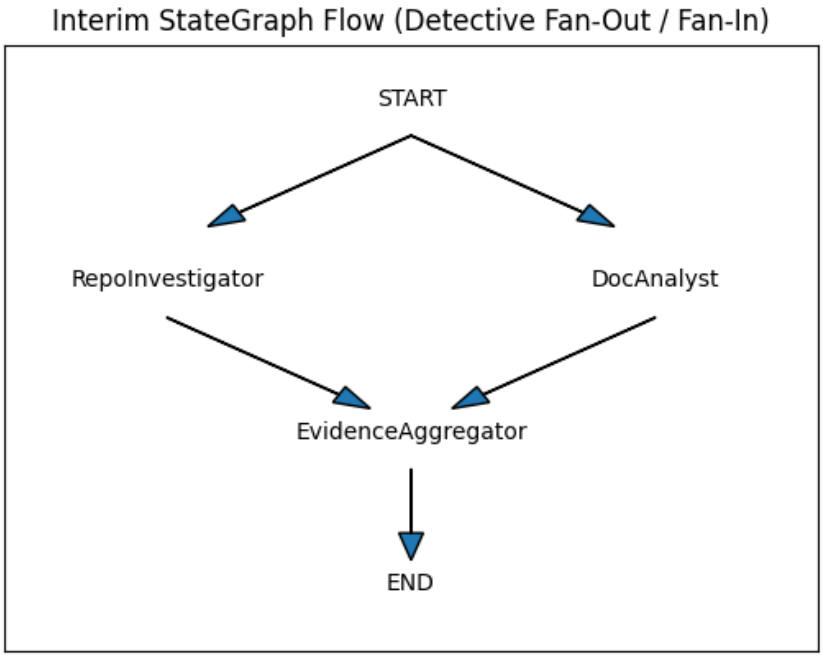
4. Planned StateGraph Flow

- Detectives Only:
  - START → RepoInvestigator || DocAnalyst → EvidenceAggregator → END

- Final Architecture:
    - START → [Detectives Parallel] → EvidenceAggregator → [Judges Parallel] → ChiefJustice → END

5. Known Gaps & Concrete Plan

- Gap 1: Structured LLM enforcement for Judges.
    - Plan: Integrate .with_structured_output(JudicialOpinion) with retry logic.
- Gap 2: Deterministic synthesis completion.
    - Plan: Implement hardcoded rule engine per rubric synthesis_rules JSON.
- Gap 3: Markdown report serialization.
    - Plan: Serialize AuditReport into Executive Summary → Criterion Breakdown → Remediation Plan.



Final Planned Digital Courtroom Architecture